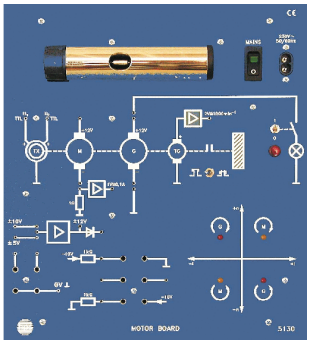
Laborversuch Regelungstechnik / Systemtechnik 2: Motorboard – Teil 2

Hochschule Esslingen
Fakultät Informationstechnik
Labor Regelungstechnik 2
Versuch: Motorboard

Bitte verwenden Sie
Matlab 2026a



Der Laborbericht ist spätestens eine Woche nach dem Labortermin abzugeben. Mangelhafte Vorbereitung beim Labortermin, verspätete oder unvollständige Abgabe des Laborberichts führen zum Nichtbestehen des Labors. Das Labor muss dann im folgenden Semester wiederholt werden.

Mit der Abgabe dieses Laborberichts und Ihrer Unterschrift bestätigen Sie, dass Sie den Bericht und die zugrundeliegenden Berechnungen, Programme und Simulationen selbständig ohne Hilfe Dritter angefertigt haben.

Laborbericht von: _____

Laborpartner: _____

Motorboard Nr.: _____ Labortermin: _____

Die Laborteilnehmer sollten jeweils ihre **eigene handschriftlich** ausgearbeitete Laborvorbereitung in die Laborveranstaltung mitbringen.

Beachten Sie im Labor die aushängenden Sicherheitsvorschriften und die Sicherheitsunterweisung.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Inhaltsverzeichnis

1 Einleitung **3**

2 Praktische Probleme bei der Drehzahl- und Lageregelung **4**

2.1 Positionsmessung, Rechen- und Abtastzeit 4

2.2 Messung der Sprungantworten der Regler und der Rechenzeiten 6

2.3 Haft–Gleitreibungseffekte 8

2.4 Genauigkeit der Positionsregelung 8

2.5 Positionsregelung mit PI–Regler? 9

3 Automatische C/C++–Codeerzeugung **12**

3.1 Konfiguration des Codegenerators 12

3.2 Einbau und Test des automatisch generierten Codes auf dem Mikrocontroller . . 17

4 Umstellung der Mikrocontroller–Implementierung auf Integer–Arithmetik **18**

4.1 Probleme der Integer–Arithmetik 19

4.2 Entwurf der Regler in Integer–Arithmetik 19

4.2.1 Entwurf der Reglerskalierungen im Blockschaltbild 21

4.2.2 Rundungsprobleme, Überläufe und Genauigkeit beim Drehzahlregler . . . 22

5 Anhang: Analyse von Messdaten **26**

5.1 Software-Oszilloskop 26

5.2 Bedienung des Digital–Speicher–Oszilloskops 27

6 Anhang: Bedienoberfläche des Mikrocontroller–Programms **28**

Hinweis:
Die Simulink–Modelle für diesen Laborversuch und die hier abgebildeten Diagramme wurden mit Matlab R2026a erstellt. Falls Sie eine neuere Version von Matlab verwenden, speichern Sie die Simulink–Modelle bitte in der Version R2026a und bringen Sie diese Dateien dann ins Labor mit.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

1 Einleitung

Dies ist der zweite Teil des Versuchs Motorboard. Im ersten Teil wurde die Regelstrecke untersucht, deren Parameter bestimmt und eine Drehzahl- und Positionsregelung entworfen. In diesem Teil wird die Regelung weiter analysiert, Möglichkeiten der automatischen C–Codegenerierung untersucht und Implementierungsaspekte bei einfachen Mikrocontrollern betrachtet.

Das (vereinfachte) Blockschaltbild der in Kaskadenstruktur aufgebauten Positions- und Drehzahlregelung ist in Abb. 1 dargestellt. Die vollständige Beschreibung der Regelstrecke, des Simulationsmodells, des Mikrocontroller–Programms sowie Hinweise zur Labordurchführung und zum Laborbericht finden Sie im Umdruck zum ersten Teil des Labors.

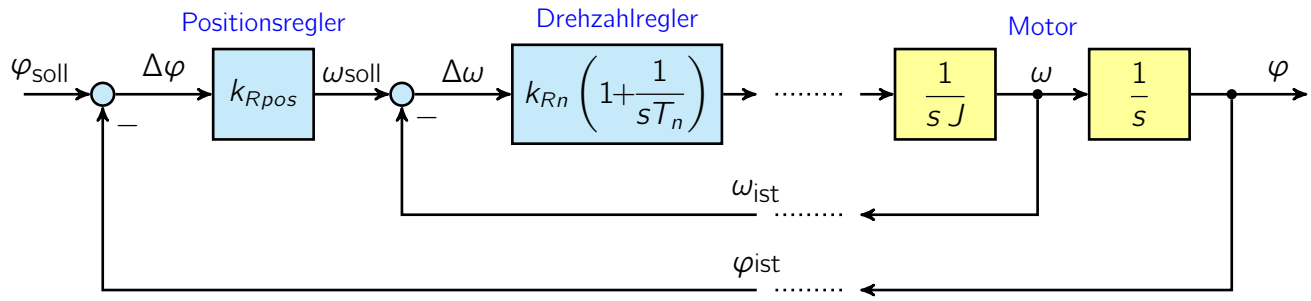


Abbildung 1: Positions- und Drehzahlregelkreis (stark vereinfacht)

Die Reglerparameter, die wir im ersten Teil des Laborversuchs ermittelt haben, führen zu relativ aggressivem Regelverhalten. Daher reduzieren wir die Parameter im Folgenden deutlich.
Bitte verwenden Sie zur Implementierung der Regler unabhängig von Ihren Ergebnissen aus dem ersten Teil des Labors in der Datei *params_m.m* die folgenden Werte:

$$\text{Drehzahlregler: } k_{Rn} = 0.075 \text{ und } T_{Nn} = 51.1ms$$

$$\text{Positionsregler: } k_{Rpos} = 5.3$$

Die Abtastzeit der Regler bleibt für Teil 2 auf $T = 1000\mu$ sec eingestellt.
Tragen Sie diese Werte bitte in die Dateien *params_m.m* und *main.c* ein!

Vorbereitungsaufgabe 1: Simulation der Drehzahl- und der Positionsregelung

Simulieren Sie mit diesen Reglerparametern im Simulink–Modells *Regelkreis_ZK.slx* die Sprungantworten ...

- ... der Drehzahlregelung bei einem Sprung der Solldrehzahl von 1000 auf $3000U/min$,
- ... der Positionsregelung bei einem Sprung der Sollposition von 0 auf 10 U.

Zoomen Sie so in die Sprungantworten hinein, dass der Einschwingvorgang bei der steigenden Flanke des Drehzahl- bzw. Positionswertes jeweils bildfüllend zu sehen ist (Zeitbereich

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

ungefähr 0,9 ... 1,3 sec). Zu dieser Aufgabe müssen Sie nichts abgeben.

2 Praktische Probleme bei der Drehzahl- und Lageregelung

2.1 Positionsmessung, Rechen- und Abtastzeit

Zur Positionserfassung verwenden wir den Positionsgeber TX auf dem Motorboard, der zwei um 90° versetzte Rechtecksignale U_1 und U_2 mit je 100 Pulsen je Umdrehung, insgesamt also 400 Signalflanken je Umdrehung, erzeugt (vergl. Abschnitt 3 im ersten Teil des Laborversuchs). Die Positionserfassung erfolgt durch eine Statemachine, die in einer der PIOs des RP2040 in Assembler programmiert ist und in der Hardware nebenläufig zum eigentlichen Prozessor ausgeführt wird. Die Positionserfassung in der PIO–Statemachine erfolgt wie folgt:

- Die Signale U_1 und U_2 vom Quadratur–Positionsgeber TX werden einsynchronisiert. Das erfolgt durch eine Kette hintereinander geschalteter D–Flip–Flops. Dies ist notwendig, da diese Signale asynchron zum Takt des Mikrocontrollers sind und es sonst zu Metastabilität kommen könnte.
- Die Statemachine in der PIO liest die beiden Signale in einem relativ schnellen Takt ein. Falls sich die aktuell eingelesenen Signale im Vergleich zu denjenigen im vorherigen Taktzyklus nicht geändert haben, bleibt die Statemachine in ihrem aktuellen Zustand und der Step–Zähler bleibt unverändert. Falls beispielsweise eine Veränderung des Eingangssignalbündels $[U_1, U_2]$ von $[0, 1]$ auf $[1, 1]$ erkannt wird (Vergleiche Abbildung 4 auf Seite 11 des ersten Laborteils) ist eine positive Flanke des Signals U_1 bei einem High–Pegel des Signals U_2 erfolgt. Die Welle hat sich also um einen Step vorwärts gedreht und die Statemachine inkrementiert den Step–Zähler. Damit liegt im Step-Zähler immer die aktuelle Position der Welle vor.

Diese vereinfachte Darstellung trifft auf einen einfachen Encoder zu. Im Labor verwenden wir allerdings eine verbesserte Version, den Substep–Encoder. Dieser weist folgende Verbesserungen auf:

- Die Steps werden Raw–Steps genannt und feiner unterteilt in Substeps; 64 Substeps je Raw–Step.
- Bei jeder Veränderung des Eingangssignalbündels $[U_1, U_2]$ wird ein Zähler initialisiert und bei jedem Durchlauf durch die Statemachine dekrementiert. Dadurch kann die verstrichene Zeit zwischen der letzten Änderung des Step–Zählers und der momentanen Abfrage ermittelt werden.
- Durch die Kenntnis der Zeitpunkte der Veränderungen des Eingangssignalbündels $[U_1, U_2]$ und der Steps kann eine sehr genaue Drehzahlerfassung stattfinden.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

In der *repeating_timer_callback()*–Funktion wird der Step–Zähler der Statemachine ausgelesen. Da dieser Zäher lediglich mit 32–bit Werten arbeitet, wird die Positionsinformation in einen 64–bit Integer akkumuliert. Die Drehzahl wird in der Variablen *rpm_substepencoder* zur Verfügung gestellt wird.

Vorbereitungsaufgabe 2: Berechnung von Zeiten zur Positionsdekodierung

Wir wollen den Zeitabstand zwischen zwei Signalflanken des Positionsgebers bei einer Motordrehzahl $n_0 = 7800U/min$ (Leerlaufdrehzahl des Motors bei Nennspannung) ermitteln. Dazu ermittel wir die folgenden Größen:

Dauer einer Motorumdrehung: $T_{rot} =$

Anzahl Signalflanken je Umdrehung: N = 400 1/Umdrehung

Zeitabstand der Signalflanken: $T_{encoder} =$

Alternativ zum Encoder in einer PIO des RP2040 könnte die Verarbeitung auch mit Hilfe von Interrupt–Routinen erfolgen. Allerdings würde dies den Prozessor bei hohen Drehzahlen sehr stark belasten, da dann der Prozessor alle $T_{encoder}$ unterbrochen werden würde.

Alternativ könnte man die Verarbeitung der Encoder–Signale auch in eine separate Hardware, z.B. ein FPGA auslagern. Beim Entwurf des RP2040 wurde entschieden, neben den üblichen standard Schnittstellen (z.B. SPI, I2C, UART, etc.) auch die Möglichkeit der flexiblen Verarbeitung von weiteren Schnittstellen in Hardware–Statemachines zu ermöglichen, wovon wir im Labor Gebrauch machen.

Da unsere Regelung mit einer relativ kleinen Abtastzeit arbeitet, sollte sich das Verhalten einer Quasi–zeitkontinuierlichen Regelung ergeben. Bei der Auslegung des Positionsreglers und bei der Dimensionierung des Drehzahlreglers im ersten Teil des Laborversuchs haben wir jeweils eine gewünschte Phasenreserve von 80° bzw. 60° vorgegeben. Dass die Regler anschließend zeitdiskret implementiert werden, haben wir dabei nicht berücksichtigt. Dies ist zulässig, solange sich die Phasenreserve bei der Durchtrittskreisfrequenz ω_D, die sich bei der dortigen Auslegung ergeben hat, durch die Abtastung nicht wesentlich reduziert.

Vorbereitungsaufgabe 3: Überprüfung der Phasenränder der beiden Regler

Berechnen Sie sowohl für den Positionsregler als auch für den Drehzahlregler die Verringerung der Phasenreserve bei der von uns gewählten Abtastzeit $T = 1000\mu$ sec sowie die sich dadurch tatsächlich zu erwartende Phasenreserve.

Sie sollen hierfür die Ergebnisse verwenden, die Sie im ersten Laborversuch ermittelt haben, da wir die Durchtrittskreisfrequenzen nur für diese Werte ermittelt haben.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Durchtrittskreisfrequenz **Positionregler** $\omega_{D,pos} =$

$\Rightarrow$ Verringerung der Phasenreserve $\Delta\varphi_{R,pos} =$

$\Rightarrow$ Resultierende Phasenreserve $\varphi_{R,pos} =$

Durchtrittskreisfrequenz **Drehzahlregler** $\omega_{D,rpm} =$

$\Rightarrow$ Verringerung der Phasenreserve $\Delta\varphi_{R,rpm} =$

$\Rightarrow$ Resultierende Phasenreserve $\varphi_{R,rpm} =$

Aus diesen Ergebnissen ist auch ersichlich, weshalb sich im ersten Teil des Laborversuchs relativ große Änderungen zwischen der Simulation im zeitkontinuierlichen und dem wirklichen Verhalten der Regelung mit dem Mikrocontroller ergeben haben.

2.2 Messung der Sprungantworten der Regler und der Rechenzeiten

Die Regelung kann nur dann sinnvoll funktionieren, wenn die Rechenzeit der Reglerfunktionen einschließlich der notwendigen Analog–Digital–Wandlungen (deutlich) kleiner ist als die gewählte Abtastzeit. Als nächstes sollen daher die einwandfreie Funktion der Regelkreise am Mikrocontroller–Board überprüft und die Rechenzeiten gemessen werden.

Laboraufgabe 1: Überprüfen der Drehzahlregelung und Rechenzeit des Drehzahlreglers

Verwenden Sie Ihr Mikrocontroller–Projekt *Lab1_uC*, in dem Sie im ersten Teil des Laborversuchs bereits den PI–Drehzahlregler mit Anti–Windup–Massnahme eingebaut haben. Stellen Sie die Reglerparameter auf die im Kasten auf S.3 angegebenen Werte ein, laden Sie das Projekt auf das Mikrocontroller–Board und testen Sie im Automatik–Betrieb der **Drehzahlregelung** Drehzahlsprünge zwischen 1000 und 3000 $1/min$. **Wenn die Dioden der Vierquadrantenanzeige wild blinken, schalten Sie das Board bitte möglichst schnell wieder aus** und überprüfen Sie Ihre Implementierung.

Messen Sie den Drehzahlverlauf mit Hilfe des Software–Oszilloskops (sh. S.26). Im Mikrocontrollerprogramm wird am Ende der Routine `repeating_timer_callback()` bei der Dreh-

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

zahlregelung die Funktion

```
SoftScopeSend((int16_t)set_rpm, (int16_t)rpm_substepencoder);
```

aufgerufen. Die Funktion überträgt bei jedem Abtastzeitpunkt die Soll–Drehzahl und die Drehzahl vom Encoder jeweils als 16bit–Integerwert an den PC. Die Werte können in Matlab mit dem Skript `SoftScope()` angezeigt werden (sh. S.26). Zoomen Sie so in das Bild hinein, dass Sie einmal die steigende und einmal die fallende Drehzahlflanke mit Ihren Simulationsergebnissen aus **Vorbereitungsaufgabe 1** vergleichen können.

Zum Vergleich der Drehzahlen des Encoders und des Tachogenerators kommentieren Sie die Zeile

```
SoftScopeSend((int16_t)set_rpm, (int16_t)rpm_substepencoder);
```

aus und übertragen Sie statt der Soll–Drehzahl die Drehzahl, die vom Tachogenerator ermittelt wird (*measured_rpm_tacho*). Fügen Sie dieses Bild Ihrem Laborbericht hinzu. Machen Sie danach die Änderung wieder rückgängig, so dass wieder wieder die Soll–Drehzahl und die gemittelte Drehzahl vom Encoder an den Entwicklungs–PC geschickt werden.

Messung der Rechenzeit des Drehzahlreglers
Die Rechenzeit des Drehzahlreglers wird mit Hilfe der Routine `time_us_64()` erfasst und im LCD–Display in der zweiten Zeile angezeigt, wenn der automatische Stimulusgenerator aktiviert ist. Die gemessenen Werte enthalten die Rechenzeit des Reglers inklusive ADC–Wandlungen und Stimulusgenerierung. Messen Sie die Rechenzeiten für die Drehzahlregelung und tragen Sie den Werte in die linke Spalte in der Tabelle auf Seite 25 ein.

Als Positionsregler haben Sie im ersten Teil des Laborversuchs einen **P–Positionsregler** in Ihr Programm eingebaut. Der Positionsregler sollte sich in der Funktion *positionControl()* in *main.c* befinden. Die Reglerverstärkung soll hier und in allen folgenden Aufgaben wieder auf die im Kasten auf S. 3 angegebenen Werte eingestellt werden. Der Ausgang des Positionsreglers, d.h. die Solldrehzahl für den Drehzahlregler, muss auf $\pm 7800 U/min$ begrenzt sein.

Laboraufgabe 2: Messungen am Positionsregler und Vergleich mit der Simulation

Schalten Sie nun über die Bedientasten auf die **Positionsregelung** um und stellen Sie den Automatik–Betrieb ein, bei dem Sollpositionssprünge zwischen 0 und 10 U (Umdrehungen) gemacht werden. **Wenn die Dioden der Vierquadrantenanzeige wild blinken, schalten Sie das Board bitte möglichst schnell wieder aus** und überprüfen Sie Ihre Implementierung.

Wir wollen nun die Sprungantworten messen und die gemessene Position und die Drehzahl der Welle mit der Simulation vergleichen. Zur Messung wird wieder das Software–Oszilloskop verwendet. Im Mikrocontrollerprogramm wird am Ende der Routine `repeating_timer_callback()` bei der Positionsregelung die Funktion

```
SoftScopeSend((int16_t) current_substep_int64, (int16_t)rpm_substepencoder);
```


© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

aufgerufen. Die Funktion überträgt bei jedem Abtastzeitpunkt die Ist–Position und die Ist–Drehzahl als 16bit–Integerwert an den PC. Die Werte können in Matlab mit dem Skript `SoftScope()` angezeigt werden (sh. S.26).

Nehmen Sie die gemessenen Zeitverläufe in Ihren Laborbericht auf und vergleichen Sie die hier gemessenen Zeitverläufe für Position und für die Drehzahl mit den Simulationsverläufen aus der Vorbereitungsfrage 1. Falls die Verläufe stark voneinander abweichen, überprüfen Sie, ob Sie bei den Simulationen und bei den Messungen tatsächlich dieselben Reglerparameter und dieselben Sollwertsprünge eingestellt haben.

Messung der Rechenzeit
Messen Sie die Rechenzeit des Positionsreglers, die im Automatik–Betrieb der Positionsregelung im LCD–Display angezeigt wird, und tragen Sie den Wert in die linke Spalte in der Tabelle auf Seite 25 ein.

2.3 Haft–Gleitreibungseffekte

In Vorbereitungsaufgabe 1 wurde das Großsignalverhalten der Positionsregelung bei einem Sollwertsprung von 0 auf 10 U mit Hilfe des Simulink–Modells *Regelkreis_ZK.slx* simuliert. Dabei wird die Drehzahl so groß, dass praktisch nur die Gleitreibung eine Rolle spielt. Bei sehr kleinen Drehzahlen dagegen macht sich auch die Haftreibung bemerkbar.

Vorbereitungsaufgabe 4: Stick–Slip–Effekt

Ändern Sie den Parameter *friction_offset*, der die Haftreibung beschreibt, auf den Wert 0.01 ab. Simulieren Sie dann wieder einen Sollwertsprung von 0 auf $10U$ zur Zeit $t_w = 1\,\text{sec}$ mit dem Simulink–Modell *Regelkreis_ZK.slx*. Sehen Sie sich die Simulationsergebnisse von Drehzahl und Position an. Sie werden feststellen, dass die Ergebnisse von denen der vorigen Aufgabe deutlich abweichen.

Der Übergang zwischen Haft- und Gleitreibung im Bereich um die Drehzahl 0 ist ein stark nichtlinearer Effekt. Dieser **Stick–Slip–Effekt** kann dazu führen, dass sich statt einer konstanten Position eine Grenzschwingungen einstellt.

Stellen Sie sicher, dass der Parameter in *params_m.m* für die folgenden Aufgaben wieder den ursprünglichen Wert besitzt.

Zu dieser Aufgabe müssen Sie nichts abgeben.

2.4 Genauigkeit der Positionsregelung

Der Positionsregler wurde im ersten Teil des Laborversuchs als P–Regler nach dem Verfahren mit Phasenreserve entworfen. Wegen des fehlenden I–Anteils im Regler soll hier die Genauigkeit der Positionsregelung analysiert werden.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Vorbereitungsaufgabe 5: Stationäre Genauigkeit des Positionsreglers

Weshalb genügt unter **idealen Verhältnissen, d.h. bei idealer Positions- und Drehzahlmessung**, ein reiner P–Regler als Positionsregler, um im eingeschwungenen Zustand eine verschwindende Regelabweichung $\Delta\varphi(t \to \infty) = 0$ sicherzustellen? Betrachten Sie zur Beantwortung dieser Frage das (vereinfachte) Blockschaltbild des Regelkreises in Abb. 1 und geben Sie die stationären Werte der unten angegebenen Signale an.

$\omega(\infty) =$ $\omega_{ist}(\infty) =$ $\Delta\omega(\infty) =$

$\omega_{soll}(\infty) =$ $\Delta\varphi(\infty) =$

Im ersten Teil des Laborversuchs in Laboraufgabe 3 haben wir festgestellt, dass die Drehzahlmessung mit dem Tachogenerator einen Messfehler verursacht, d.h.

$$\omega_{ist} = \omega_{ofs} + k_n \cdot \omega = 2\pi(n_{ofs} + k_n \cdot n).$$

Dabei entspricht $\omega_{ofs} = 2\pi\ n_{ofs} \neq 0$ einem Offsetfehler, $k_n \neq 1$ einem Steigungsfehler.

Führt der Messfehler des Tachogenerators (Drehzahlmessung) zu einer bleibenden Abweichung der Positionsregelung? Geben Sie eine Formel zur Berechung der bleibenden Regelabweichung $\Delta\varphi(t \to \infty)$ der Positionsregelung an. Die Positionsmessung selbst darf weiterhin als ideal angenommen werden, d.h. $\varphi_{ist} = \varphi$.

$\omega(\infty) =$ $\omega_{ist}(\infty) =$ $\Delta\omega(\infty) =$

$\omega_{soll}(\infty) =$ $\Delta\varphi(\infty) =$

Um die Lagereglung genauer zu machen, müsste der Offsetfehler der Drehzahlmessung kompensiert werden. Das wäre z.B. möglich, indem man den Offsetfehler misst und dann im Mikrocontrollerprogramm vom gemessenen Wert abzieht. Wenn der Offsetfehler zeitlich konstant ist, ist dies die einfachste Möglichkeit, wenn man die Drehzahl des Tachogenerators zur Regelung verwendet.

2.5 Positionsregelung mit PI–Regler?

Alternativ könnte man überlegen, den P–Positionsregler durch einen PI–Regler zu ersetzen. Wie dies die Stabilität und das dynamische Verhalten des Regelkreises beeinflussen würde, soll mit Hilfe der Wurzelortskurve untersucht werden. Der offene Positionsregelkreis (Abb. 1) enthält folgende dominierenden Pole bzw. Nullstellen, die alle reell sind (Abb. 2):

- Einen Pol bei $s_{\infty,1} = 0$ durch den Integrator zwischen Drehkreisfrequenz $\omega = 2\pi n$ und Position φ.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

- Einen Pol, der von der mechanischen Zeitkonstante des Motors im Drehzahlregelkreis resultiert, bei $s_{\infty,2} \approx -18.5\frac{\text{rad}}{\text{sec}}$
- Einen weiteren Pol bei $s_{\infty,3} \approx -120\frac{\text{rad}}{\text{sec}}$, der ebenfalls von der Drehzahlregelung stammt.
- Eine Nullstelle bei $s_0 \approx \frac{1}{T_{n_n}} \approx -20\frac{\text{rad}}{\text{sec}}$ durch den PI–Drehzahlregler. Diese Nullstelle sollte eigentlich die mechanische Zeitkonstante kompensieren, ist dafür aber bei den in diesem Teil des Labors gewählten Reglerparametern etwas zu schnell.

Die vergleichsweise kleinen Zeitkonstanten des PWM–Filters, der Drehzahlmessung sowie des elektrischen Teils des Motors (alle $\leq$ 2msec), wurden dabei vernachlässigt.

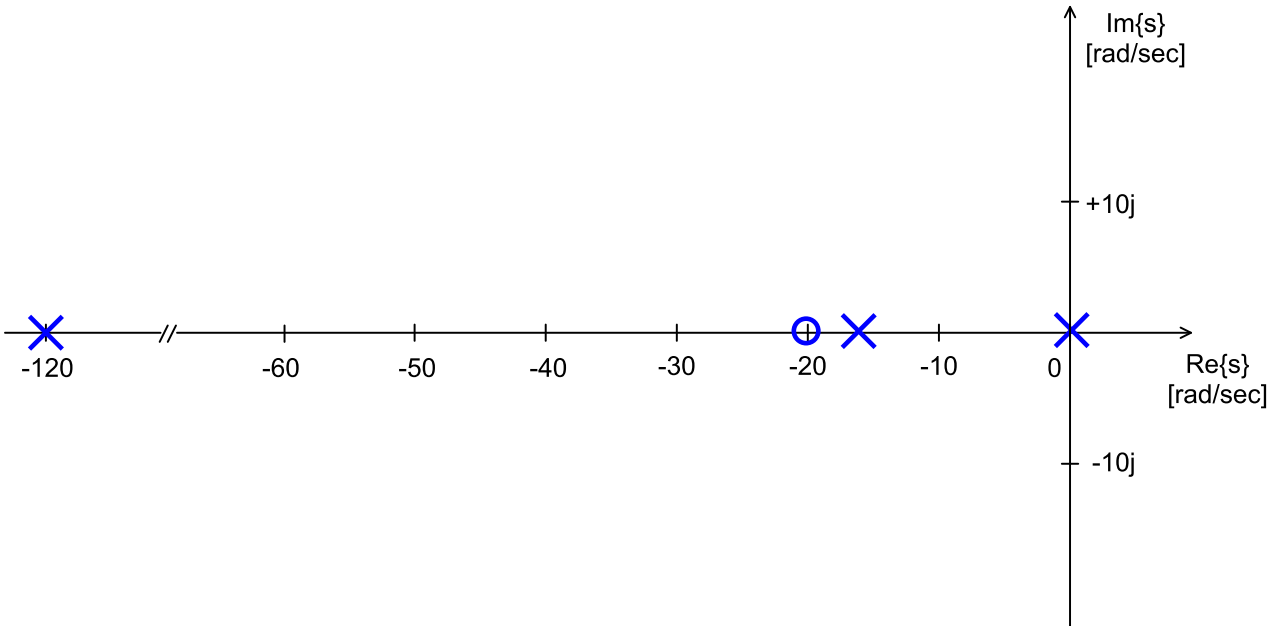


Abbildung 2: (Vereinfachte) Wurzelortskurve des Positionsregelkreises mit P–Regler

Vorbereitungsaufgabe 6: Wurzelortskurve des P–Positionsregelkreises

In Abb. 2 soll die Wurzelortskurve (WOK) des Positionsregelkreises mit P–Positionsregler skizziert werden. Die Pole und Nullstellen sind bereits eingezeichnet.

- Wieviele Äste wird die WOK haben und wieviele davon gehen ins Unendliche?

- Geben Sie den Schnittpunkt σ_a und die Winkel φ_a der Asymptoten für diejenigen Äste der WOK an, die ins Unendliche gehen und zeichnen Sie diese Asymptoten ein.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

- Markieren Sie in Abbildung 2 diejenigen Abschnitte auf der reellen Achse, auf denen Äste der WOK liegen.
  Deuten Sie durch Pfeile an, in welcher Richtung die WOK in der Umgebung der Pole bzw. Nullstelle verläuft, wenn die Reglerverstärkung $k_{R_pos} \uparrow$ des Positionsreglers vergrößert wird.
- In welchem Bereich auf der reellen Achse müssen Verzweigungs- bzw. Vereinigungspunkte der WOK liegen?

- Skizzieren Sie die vollständige WOK in Abbildung 2.

Vorbereitungsaufgabe 7: Wurzelortskurve eines PI–Positionsregelkreises

Wenn man statt des P–Positionsreglers einen PI–Regler

$$G_{R_pos} = k_{R_pos} \frac{1 + sT_{n_pos}}{sT_{n_pos}}$$

einsetzt, ergibt sich ein weiterer Pol bei $s_{\infty,PI} = 0$ und eine Nullstelle bei $s_{0,PI} = -\frac{1}{T_{n_pos}}$.

Während der zusätzliche Pol fix ist, kann die neue Nullstelle durch die Wahl von T_{n_pos} auf der negativen reellen Achse frei platziert werden, etwa

a) zwischen die beiden langsamen Pole z.B. bei $s_{0,PI} = -12.5\frac{\text{rad}}{\text{sec}}$ oder

b) zwischen die beiden schnellen Pole z.B. bei $s_{0,PI} = -25\frac{\text{rad}}{\text{sec}}$.

Verwenden Sie das Matlab–Skript *wok_Positionsregelkreis_PI.m*, um die WOK für die Dimensionierungen a) bzw. b) zu zeichnen.

Hinweis: Durch Modifikation dieses M–Files können Sie auch Ihr Ergebnis der vorherigen Vorbereitungsaufgabe überprüfen.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Fügen Sie beide Diagramme für a) und b) in Ihren Laborbericht ein. Sehen Sie eine Chance, mit dem PI–Positionsregler einen stabilen Regelkreis aufzubauen (Begründung)? Wird die Sprungantwort überschwingen?

3 Automatische C/C++–Codeerzeugung

Die modellbasierte Vorgehensweise mit Werkzeugen wie Matlab/Simulink erlaubt eine schnelle Systementwicklung mit hohem Abstraktionsgrad. Die anschließende manuelle Umsetzung in einen echtzeitfähigen Embedded–C/C++–Programmcode kann bei komplexen Aufgabenstellungen allerdings immer noch langsam, unflexibel und fehlerträchtig sein. Daher wird zunehmend versucht, diesen Schritt durch automatisierte Codegenerierung zu ersetzen.

3.1 Konfiguration des Codegenerators

Um diese Vorgehensweise kennenzulernen, verwenden wir das Simulink–Projekt **Regelkreis_ZD.slx**. Dieses Projekt enthält wiederum die Positions– und Drehzahlregelung für das Motorboard, wobei in beiden Reglern nun zeitdiskrete Simulinkblöcke eingesetzt werden und beide Regler jeweils in ein sogenanntes Subsystem *posRegler* bzw. *nRegler* verschoben wurden.

Bevor Sie das Modell simulieren oder daraus C/C++–Programmcode erzeugen, müssen Sie wieder das Matlab–Skript *params_m.m* ausführen, das die Parameter für das Modell einstellt. **Vergewissern Sie sich, dass hier die gewünschten Reglerparameter eingestellt sind, da diese hart in den generierten C–Code geschrieben werden!**

Laboraufgabe 3: Blick unter die Maske

Das „Innenleben“ eines Subsystems können Sie sich durch Doppelklick ansehen. Gehen Sie in das Subsystem *nRegler* und klicken Sie den Block *zPI_AWU1* mit der rechten Maustaste an. Wählen Sie dann im Menü den Punkt *View Mask* und dann *Look Inside Mask*. Sie sehen dann das Innenleben des Blockes. Sehen Sie sich dieses Innenleben an. Anhand dieser Darstellung können Sie später dann auch den generierten Code besser verstehen.

Simulieren Sie nochmals den Drehzahl- und den Positionsregler um sicherzustellen, dass alles im Modell richtig eingestellt ist.

Wir wollen nun aus dem Modell den C/C++–Programmcode für die beiden Regler automatisch generieren. Der Simulink–Codegenerator lässt sich in weiten Grenzen konfigurieren.

Hinweis: Diese Beschreibung und die Screenshots wurden mit **Matlab R2026a** erstellt, andere Versionen können davon abweichen!

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Laboraufgabe 4: Codegenerierung für den Positions- und den Drehzahlregler

- Öffnen Sie das Simulink–Modell *Regelkreis_ZD.slx* (wahrscheinlich haben Sie das schon offen).
- Öffnen Sie das Fenster *Model Settings* über **CTRL+E** (alternativ im Tab *Modeling* und dann *Model Settings Setup*).
- Unter *Solver* (Abb. 3) stellen Sie einen *Fixed–step* Solver ein und legen die Integrationsschrittweite (bei *Solver details*) auf unsere Abtastzeit T fest.
- Unter *Hardware Implementation* (Abb. 4) muss der Mikrocontrollertyp *Raspberry Pi* ausgewählt werden.
- Unter *Code Generation* (Abb. 5; hier sind nur die Einstellungen zu überprüfen) wird die Programmiersprache C sowie die Zielumgebung *Embedded Realtime Target (ERT.tlc)* ausgewählt. Der erzeugte Code soll ANSI kompatibel sein. Die gewählte Tookchain hängt vom Betriebssystem ab.
- Unter *Interface* (Abb. 6) wird ausgewählt, dass *Floating–Point*–Variablen gewünscht sind und auch Blöcke mit kontinuierlicher Zeitabhängigkeit unterstützt werden sollen. Entfernen Sie den Hacken bei *Remove error status field in real–time model data structure*. Der generierte Code soll im Wesentlichen aus einer Intialisierungsfunktion <Name des Subsystems>_initialize() sowie einer Funktion <Name des Subsystems>_step() besteht, die bei jedem Abtastschritt aufgerufen werden muss (*Single output/update function*). Diese Optionen sind Default und Sie können sie ganz unten unter *Advanced parameters* sehen.
- Die Rubrik *Code Placement* (Abb. 7) enthält die Deklaration der Datenstrukturen (*Data definition* und *Data declaration*). Wählen Sie hier bei *Code packaging* unter *File packaging format*: die Option *Modular*. Hier wird vorgegeben, dass neben der Code–Datei eine Header–Datei erzeugt wird. Weiter unten können die Namen der erzeugten Dateien vorgegeben werden.

Nachdem diese Einstellungen vorgenommen wurden, müssen **für jedes unserer beiden Subsysteme** *posRegler* bzw. *nRegler* folgende weitere Einstellungen gemacht werden:

- klicken Sie das betrachtete Subsystem mit der rechten Maustaste an und wählen Sie die Option *Is Atomic*.
- klicken Sie das betrachtete Subsystem mit der rechten Maustaste an und
  - wählen Sie unter *Parameters* → *Block Parameters*
  - im Tab *Code Generation* wählen Sie
    - *Function packaging*: *Non-Reusable function*

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

    - *Function name options*: *Use subsystem name*
    - *File name options*: *Use subsystem name*

- klicken Sie das betrachtete Subsystem mit der rechten Maustaste an und wählen Sie *Convert to* → *Referenced Model*. Belassen Sie die Standardeinstellungen und klicken Sie auf *Convert*.
- In neuem Simulink-Modell:
  - Im Matlab Tab *APPS* (ganz oben) wählen Sie *Embedded Coder*.
  - Schließlich wählen Sie *Generate Code* → *Generate Code* (Generate Code only. Do not execute makefile.)
- Der erzeugte Programmcode sowie einige Hilfsdateien werden in einem Unterverzeichnis <Name des Subsystems>_ert_rtw abgelegt. Ausserdem erscheint ein Fenster „Code Generation Report“, in dem man sich unter „Code Interface Report“ die Aufrufschnittstelle der erzeugten C–Funktionen und unter „Generated Code - Model Files“ den Programmcode ansehen kann.

Der automatisch erzeugte Code für den Positionsregler befindet sich in den Dateien *posRegler.h* bzw. *posRegler.c*. Versuchen Sie den Code, insbesondere die Funktionen *posRegler_step()* und die erzeugten Datenstrukturen zu verstehen. Dazu ist es sehr hilfreich, den Reglerblock auch im Simulinkmodell anzusehen und auch wie weiter oben beschrieben unter die Maske zu sehen!

Öffnen Sie die Dateien *nRegler.c* und *nRegler.h* und versuchen Sie auch diesen Code und die erzeugten Datenstrukturen zu verstehen.

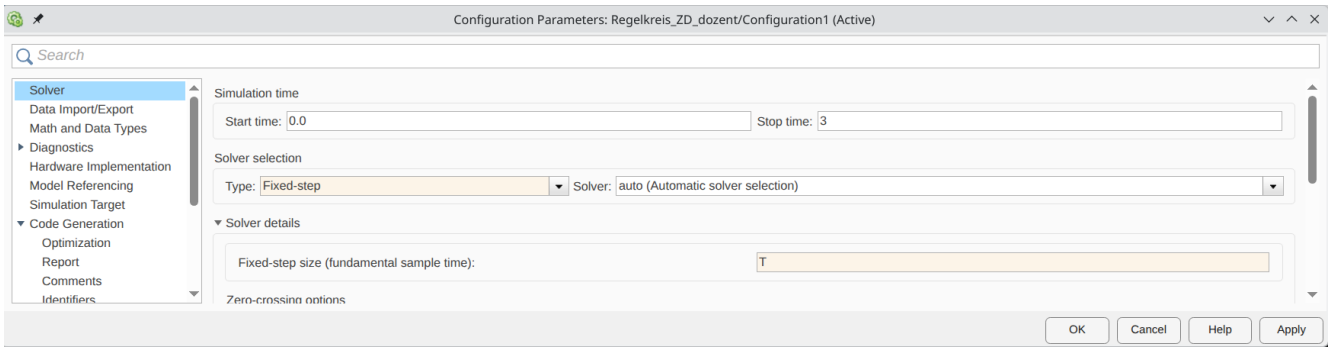


Abbildung 3: C/C++–Codeerzeugung: Zeitdiskreter Integrationsalgorithmus

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

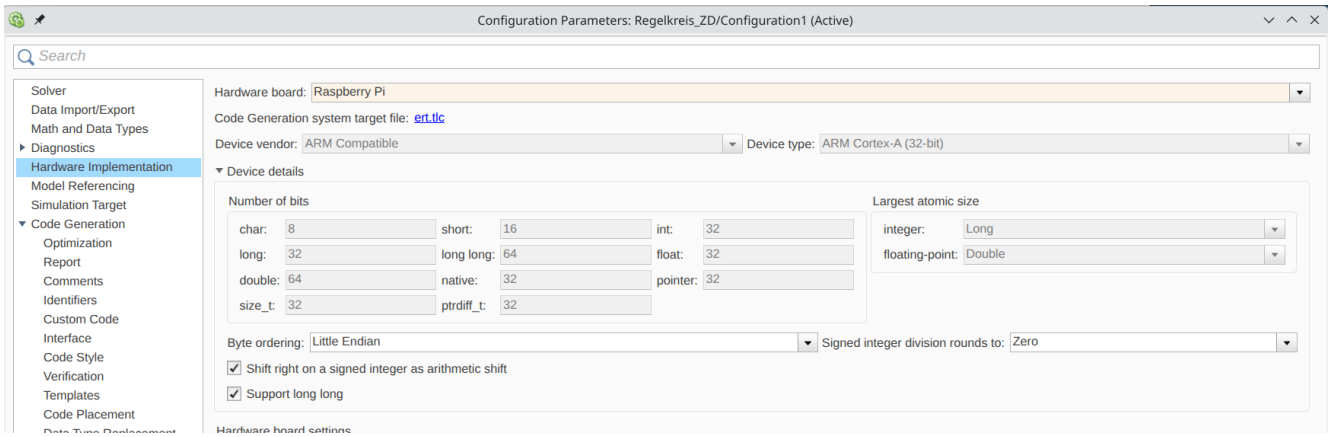


Abbildung 4: C/C++–Codeerzeugung: Hardware Implementation

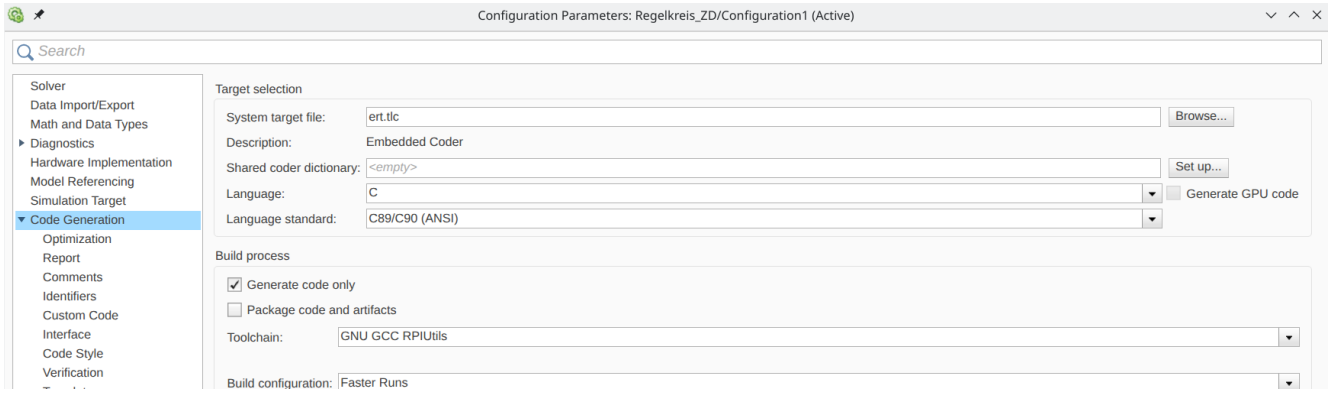


Abbildung 5: C/C++–Codeerzeugung: ERT

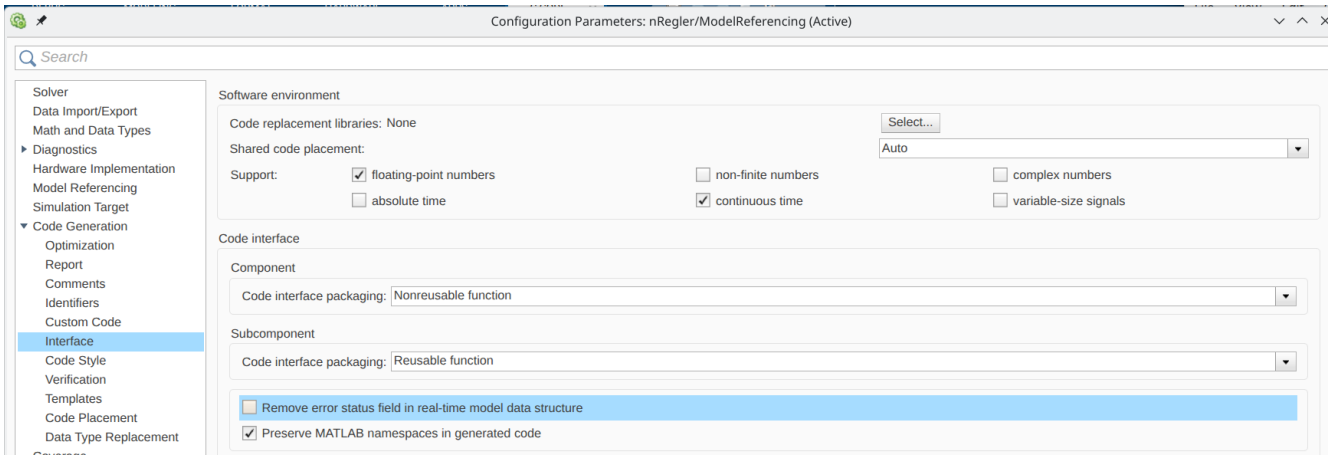


Abbildung 6: C/C++–Codeerzeugung: Interface

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

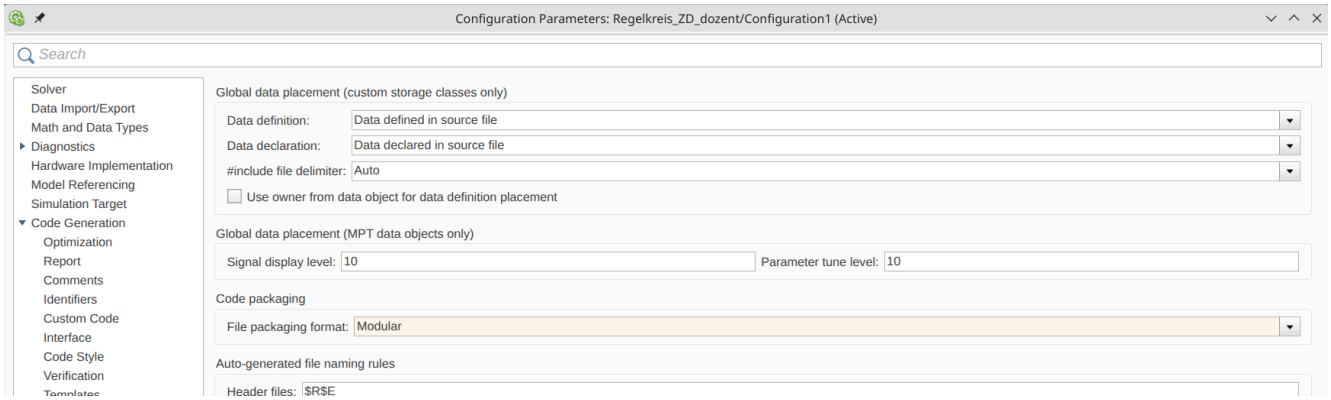


Abbildung 7: C/C++–Codeerzeugung: Code Placement

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

3.2 Einbau und Test des automatisch generierten Codes auf dem Mikrocontroller

Im Folgenden soll nun aus dem Simulink–Modell Programmcode in das Mikrocontroller–Projekt integriert werden.

Laboraufgabe 5: Einbinden des generierten Codes in das Projekt

Im nächsten Schritt muss der automatisch erzeugte Programmcode in das Mikrocontroller–Projekt *Lab2_uC_autocode* integriert werden. Der Stand dieses Projektes entspricht dem Mikrocontroller–Projekt aus dem ersten Laborversuch, bevor Sie dort begonnen haben, die Regler selbst in C zu programmieren.

- Binden Sie die vier erzeugten c–Dateien der Regler *posRegler.c*, *posRegler_data.c*, *nRegler.c*, sowie *nRegler_data.c* z.B. über Softlinks (*ln -s*) oder durch Kopieren in das *src* Verzeichnis ein.
- Binden Sie die folgenden erzeugten header–Dateien der Regler in das *include* Verzeichnis ein:
  - *nRegler.h*, *nRegler_private.h*, *nRegler_types.h*
  - *posRegler.h*, *posRegler_private.h*, *posRegler_types.h*
  - Die Header–Datei *rtwtypes.h* ist in beiden generierten Modellen enthalten. Binden Sie eine davon in das *include* Verzeichnis ein.
  - Des weiteren benötigen wir gegebenenfalls noch die beiden Header–Datein *rtw_continuous.h* und *rtw_solver.h* aus der Matlab–Installation.
- Im File *CMakeLists.txt* ergänzen Sie die vier c–Source Dateien (z.B. *src/nRegler.c*) bei *add_executable*.
- Wechseln Sie in das *build*–Verzeichnis, löschen Sie dort alles und geben dann den Befehl `cmake ..` ein.
- Öffnen Sie die Datei *main.c* und führen Sie folgende Änderungen durch:
  - Am Anfang der Datei hinter `#include "main.h";` müssen Sie die beiden Include–Dateien *posRegler.h* und *nRegler.h* einfügen.
  - In der Funktion *main()* müssen Sie vor dem Start des repeating Timers die Regler durch die Aufrufe `posRegler_initialize()` und `nRegler_initialize()` initialisieren.
  - Gehen Sie nun in die Funktion `positionControl()` und fügen dort folgende Codezeilen ein:

```
// Position controller input signals
posRegler_U.Soll = set_position;
```


© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

```
posRegler_U.Ist   = (float)current_substep_int64;
// Position controller execute (Autocode)
posRegler_step();
// Position controller output signal
set_rpm           = (float)posRegler_Y.StellB;
```

Die ersten beiden Zeilen kopieren den Soll- und den Istwert in die Eingangsdatenstruktur `posRegler_U`, in der der automatisch erzeugte Code die Eingangssignale erwartet. Die dritte Zeile ruft die Funktion auf, in der der Regler in jedem Abtastschritt berechnet wird. Die vierte Zeile kopiert die Stellgröße aus der Ausgangsdatenstruktur `posRegler_Y` zurück in unsere globale Variable.

– Analog in der Funktion `rpmControl()`:

```
// rpm controller input signals
nRegler_U.Soll = set_rpm;
nRegler_U.Ist  = rpm_substepencoder;
// rpm controller execute (Autocode)
nRegler_step();
// rpm controller output signal
rpm_control_u  = (float)nRegler_Y.StellB;
```

- Das Projekt sollte sich jetzt fehlerfrei übersetzen und auf das Mikrocontroller–Board laden lassen. Testen Sie die Drehzahlreglung und die Positionsregelung.

  Messen Sie wie in Laboraufgabe 1 die Sprungantwort des Drehzahlreglers und analog zu Laboraufgabe 2 die Sprungantwort des Positionsreglers. Falls die Sprungantworten deutlich von den Messungen in Laboraufgabe 1 und 2 abweichen, müssen Sie die Ursache suchen und beseitigen.

  Zu dieser Aufgabe müssen Sie nichts abgeben.

Messung der Rechenzeit
Messen Sie für den automatisch generierten Programmcode wiederum die Rechenzeiten in den Betriebsarten „Positionsregelung“ bzw. „Drehzahlregelung“, die im LCD–Display in der zweiten Zeile angezeigt wird, wenn der automatische Stimulusgenerator aktiviert ist. Tragen Sie diese Werte in die mittlere Spalte in der Tabelle auf Seite 25 ein.

4 Umstellung der Mikrocontroller–Implementierung auf Integer–Arithmetik

Vor allem kleinere und günstigere Mikrocontroller haben keine Gleitkomma–ALU sondern verwenden eine Softwarebibliothek, die die Gleitkommaarithmetik in Festkommaalgorithmen für die Mantissen und Exponenten zerlegt. Derartige Berechnungen können im Vergleich zu Integer–Berechnungen extrem langsam sein. Daher bietet es sich aus Kostengründen in manchen Projekten

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

an, auf Gleitkomma–Operationen zu verzichten. Aber auch bei leistungsstärkeren Mikrocontrollern führt eine Integerimplementierung im Allgemeinen zu kürzeren Laufzeiten.

4.1 Probleme der Integer–Arithmetik

Wenn aus Kostengründen keine schnellere CPU eingesetzt werden soll, muss der Algorithmus so verändert werden, dass die Berechnungen mit schnelleren Integer–Operationen erfolgen können. Dabei tauchen allerdings zwei mögliche Probleme auf:

- **Überläufe**: Während bei Additionen $a + b$ Überläufe durch das Auswerten der Carry- bzw. Overflow–Flags oder durch entsprechende Sättigungsalgorithmen erkannt und dadurch sinnvoll behandelt werden können, lassen sich Überläufe bei Multiplikationen praktisch nur vermeiden, indem man für das Ergebnis die doppelte Breite der Faktoren vorsieht, also z.B. für `int a,b; long c = (long) a * (long) b`. Dieses Vorgehen ist offensichtlich auf wenige Multiplikationen beschränkt. Glücklicherweise erfolgt in linearen Systemen keine Multiplikation zweier Signale, sondern nur die Multiplikation eines Signals mit einer Konstanten. Dadurch kann durch Begrenzung des Signals bei bekannter Konstante ein Überlauf des Zahlenbereichs vermieden werden.

- **Rundungsprobleme**: Bei Reglern sind die Zahlenwerte häufig sehr klein. Der Verstärkungsfaktor des I–Anteils ist fast immer $k_I = k_P * T/T_n \ll 1$, da die Abtastzeit $T \ll T_n$ gewählt werden muss und nur selten $k_P \gg 1$ sein dürfte. Bei Integer–Arithmetik entspricht eine Multiplikation mit $k_I < 1$ jedoch einer Multiplikation mit 0, d.h. der I–Anteil ist praktisch nicht vorhanden.

  Rundungsprobleme treten auch ganz generell auf. Falls sich bei der Reglerauslegung ein Wert von z.B. $k_P = 1.5$ ergeben hat, hat man bei direkter Integer–Umsetzung ohne Umskalierung lediglich die Wahl, mit $k_P = 1$ oder $k_P = 2$ zu arbeiten. Die Abweichung vom theoretisch berechneten Wert und damit die Auswirkung auf das Regelverhalten ist erheblich.

  Rundungsprobleme treten bereits bei der Berechnung der Regeldifferenz e auf. Sobald die Differenz bei Annäherung an den eingeschwungenen Zustand $|e| < 1$ wird, rundet die Integer–Arithmetik die Differenz auf 0 ab (je nach Implementierung kann auch aus -0.8 z.B. -1 oder 0 werden), was sowohl den P- als auch den I–Anteil bei diesen kleinen Werten faktisch abschaltet. Das „Abschalten“ führt dann dazu, dass die Regeldifferenz wieder über die Rundungsschwelle hinaus ansteigt und der Regler wieder wirksam wird, wodurch die Regeldifferenz wieder unter die Rundungsschwelle sinkt. Der Regelkreis geht dann in eine Grenzschwingung geringer Amplitude, aber hoher Frequenz über.

4.2 Entwurf der Regler in Integer–Arithmetik

Um zu einer Integer–Realisierung zu kommen, betrachtet man alle Signale und Parameter des Reglers und skaliert diese so, dass zum einen die Genauigkeit ausreichend ist und zum anderen Zahlenbereichsüberläufe so weit wie möglich vermieden werden. Falls die Vermeidung von

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Überläufen nicht garantiert werden kann, verwendet man Sättigungsalgorithmen, die im Falle eines Überlaufes in die jeweilige Begrenzung laufen statt einen Wrap–Around zu produzieren. Ein solcher Zahlenbereichsüberlauf ist insbesondere in der Regelungstechnik kritisch, da sich dadurch der Regelsinn verkehrt und der Regelkreis instabil werden kann. In der Analogtechnik findet ein solcher Wrap–Around prinzipiell nicht statt, weil ein übersteuerter Verstärker in der jeweiligen Begrenzung verharrt.

Ein Wrap–Around tritt z.B. dann auf, wenn zu einer 4–bit Zahl $+7 = (0111)_2$ die Zahl 2 addiert wird. Hier ergibt sich die Zahl $9 = (1001)_2$. Falls nun aber keine Dualcodierung sondern eine Zweierkomplement–Codierung zugrundeliegt, ergibt sich hier als Ergebnis $(1001)_z = -7$. Statt einer sehr großen positive Stellgröße wird tatsächlich eine sehr große negative Stellgröße ausgegeben. Dies kann zu katastrophalem Fehlverhalten des Systems führen.

Zahlenbereichsüberlauf: Die Möglichkeit eines Zahlenbreichtsüberlaufes muss bei allen Rechenoperationen geprüft werden. Hierfür eignet sich vor allem die Assembler–Programmierung. In Hochsprachen kann man einen Überlauf bei Addition/Subtraktion relativ einfach anhand der Vorzeichen der Operanden und des Ergebnisses erkennen. Z.B. tritt ein Überlauf auf falls $(+) + (+) = (-)$. In einem solchen Fall kann das „richtige“ Ergebnis nicht im vorgegebenen Zahlenbereich dargestellt werden. Man kann dann entweder die Bitbreite der entsprechenden Variablen erhöhen oder, falls dies nicht möglich oder gewünscht ist, als Ergebnis die größte positive Zahl verwenden, die in der zugrundeliegenden Zahlencodierung noch dargestellt werden kann. Eine solches Vorgehen bezeichnet man als Sättigung.

Laboraufgabe 6: Sättigungsalgorithmus bei Addition

Ein möglicher Sättigungsalgorithmus für die Addition zweier `int` Zahlen ist:

```
int sat_add_int( int a, int b)
{
 int s = a + b;
 if( a > 0 && b > 0 && s < 0){
     s = INT_MAX;
 }else if( a < 0 && b < 0 && ( s > 0 || s == INT_MIN) ){
     s = -INT_MAX;
 }
 return(s);
}
```

Dabei fällt auf, dass die Zahlenbereichsgrenzen nicht hart codiert sind, sondern Symbole verwendet werden. Wo sind diese in C definiert? Weshalb schließt man offensichtlich die Zahl `INT_MIN` vom darstellbaren Zahlenbereich aus? Überlegen Sie sich insbesondere, was bei der Komplementbildung von `INT_MIN` entsteht.

Definition in:

`INT_MIN` ausgeschlossen weil:

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Bei **Multiplikation** ist das Erkennen von Überläufen schwieriger. Daher schließen wir solche Überläufe in linearen Systemen algorithmisch aus, indem der Zahlenwert des variablen Operanden vor der Mulitplikation mit der bekannten Reglerkonstanten durch Sättigungselemente begrenzt wird. Die Begrenzung ist so zu wählen, dass garantiert werden kann, dass das Ergebnis der Multiplikation in der gewählten Zahlencodierung darstellbar ist.

4.2.1 Entwurf der Reglerskalierungen im Blockschaltbild

Um zu einer Integer–Realisierung zu kommen, wird das Blockschaltbild des Reglers umgeformt, indem wir an passenden Stellen die Identität $2^{\nu} \cdot 2^{-\nu}$ einfügen. Weil das Vertauschen von Blöcken und die weiteren Blockschaltbildumformungen, die Sie in der Vorlesung kennengelernt haben, bei linearen Systemen Äquivalenzumformungen darstellen, können wir die beiden Blöcke dann durch das Blockschaltbild verschieben (Kommutativgesetz) und so beispielsweise andere Blöcke des originalen Blockschaltbildes zwischen sie bekommen. Durch Anwendung des Assotiativgesetzes können wir die Blöcke auch zu neuen Blöcken zusammenfassen. Diese Umformungen sind sinnvoll, weil das System durch die nachträgliche Veränderung der Zahlendarstellung nichtlinear wird und sich somit erhebliche Änderungen im Verhalten ergeben. Dies sei an folgendem Beispiel erläutert (mit $e = 10$ und $k = 0.5$):

$$
\begin{aligned}
e \cdot k &= u \\
\text{in float} \quad \Rightarrow 10 \cdot 0.5 &= 5 \\
\text{in Integer} \quad \Rightarrow 10 \cdot 0.5 &= 10 \cdot 0 = 0 \\
\text{jetzt mit Skalierung} \quad \Rightarrow e \cdot k \cdot 2^{\nu} \cdot 2^{-\nu} &= u \\
(e \cdot (k \cdot 2^{\nu})) \cdot 2^{-\nu} &= u \\
\text{in Integer mit } \nu = 1 \quad \Rightarrow & \\
(10 \cdot (0.5 \cdot 2)) \gg 1 &= (10 \cdot 1) \gg 1 \\
&= (0 \ldots 01010)_2 \gg 1 = (0 \ldots 0101)_2 = 5
\end{aligned}
$$

Als Skalierungsfaktoren verwendet man meist Zweierpotenzen, da die Intergerdivision durch die Skalierungskonstante dann durch Schiebeoperationen zeitgünstig ausgeführt werden kann (siehe Beispiel oben: Multiplikation mit 2^{-1} entspricht Division durch 2 und wird durch arithmetisches Rechtsschieben um ein Bit implementiert).

Es ist zu beachten, dass die Gleitkommamultiplikation (hier: $0.5 \cdot 2$) vorab auf dem Entwicklungsrechner stattfindet und in der Integer–Realisierung das Ergebnis dieser Multiplikation als P–Konstante auftritt.

Abbildung 8 zeigt das Blockschaltbild des Positionsreglers mit unterlagertem Drehzahlregler und den entsprechenden Skalierungsfaktoren. Im Blockschaltbild a) sind die Skalierungsfaktoren als Identitäten eingebracht. Im darunterliegenden Blockschaltbild b) sind die Skalierungsfaktoren jeweils einem Reglerparameter zugeschlagen und die Integration (bzw. die Summenbildung im Zeitdiskreten) mit dem Block n_I^{-1} vertauscht. Im Blockdiagramm c) sollen Sie in einer folgenden Aufgabe aus der Analyse der Implementierung der Regler die Zahlenwerte der Skalierungsfaktoren

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

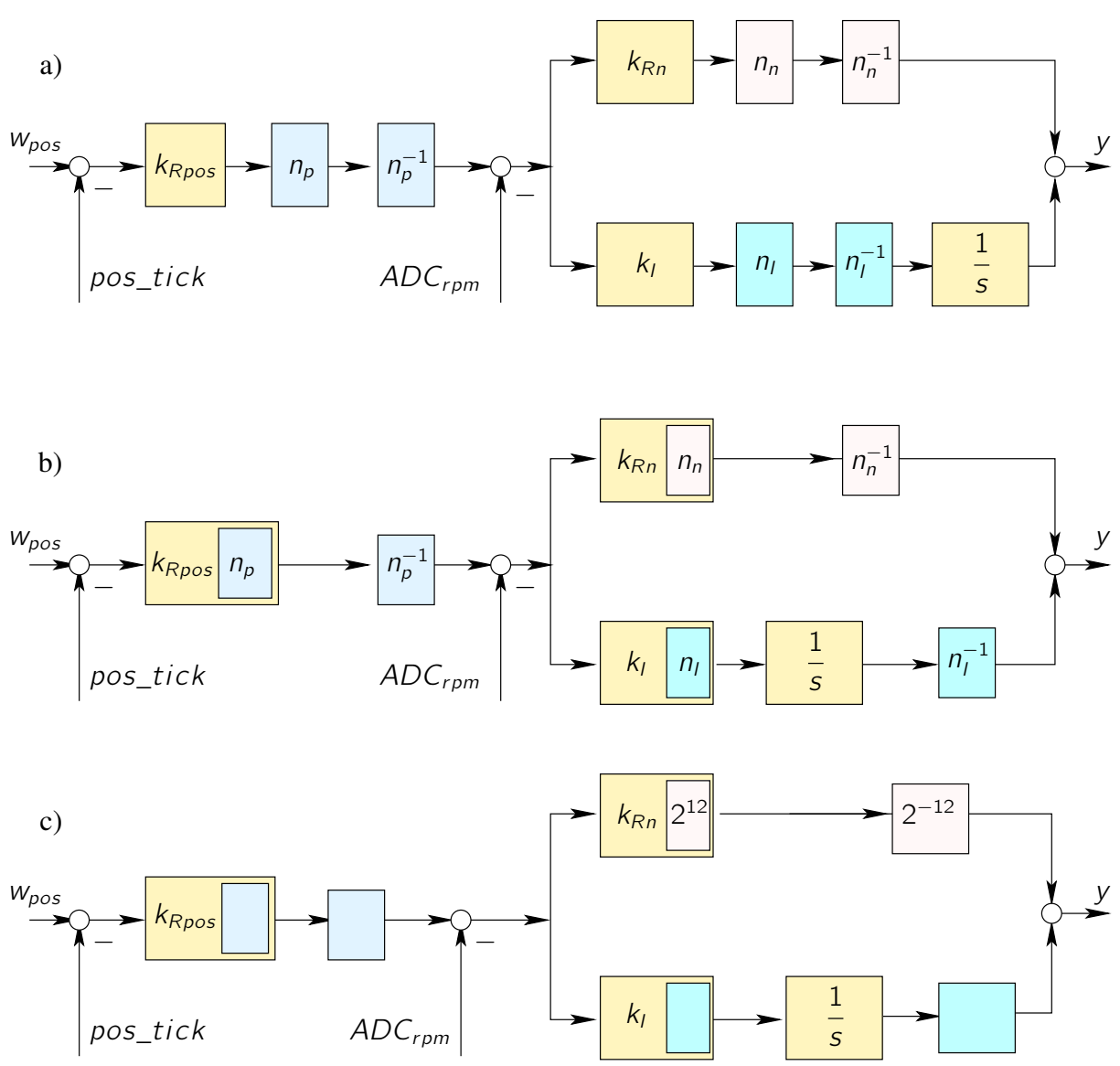


Abbildung 8: Blockschaltbildumformungen des verwendeten Positionsreglers mit unterlagertem Drehzahlregler und Skalierungsblöcken (siehe Laboraufgabe 7)

ermitteln und entsprechend eintragen. Wie oben angedeutet muss auf Genauigkeit und Zahlenbereichsüberläufe geachtet werden. Dies wollen wir im nächsten Abschnitt am Beispiel des P–Anteils des Drehzahlreglers betrachten.

4.2.2 Rundungsprobleme, Überläufe und Genauigkeit beim Drehzahlregler

Rundungsprobleme Beim gewählten Wert von k_{Rn} und dem zu wählenden Skalierungsfaktor n_n muss untersucht werden, welcher Zahlenwert sich in der Integer–Realisierung von $k_{Rn} \cdot n_n$ ergibt und ob die sich ergebende Genauigkeit ausreichend ist.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Der sich ergebend Regelparameter in der Integer–Realisierung ist $\hat{k}_{Rn} = \lfloor k_{Rn} \cdot n_n \rfloor \cdot n_n^{-1}$ ($\lfloor x \rfloor$ ist die größte ganze Zahl kleiner oder gleich x).

Mit den vorliegenden Zahlenwerten $k_{Rn} = 0.075$ und der Wahl $n_n = 2^{12}$ ergibt sich:

$$\lfloor k_{Rn} \cdot n_n \rfloor = \lfloor 0.075 \cdot 2^{12} \rfloor = \lfloor 307.2 \rfloor = 307 \qquad \Rightarrow \qquad \hat{k}_{Rn} = 307 \cdot 2^{-12} = 0.0749512$$

Es ergibt sich ein absoluter Fehler von $0.0749512 - 0.075 = -0.00004883$ was einem sehr kleinen relativen Fehler entspricht. Wir betrachten diese Genauigkeit als ausreichend.

Überläufe Mit welchem Datentypen soll die Stellgröße des P–Anteils des Drehzahlreglers (Signal im P–Pfad des Drehzahlreglers zwischen den Blöcken $\lfloor k_{Rn} \cdot n_n \rfloor$ und n_n^{-1} Abbildung 8 b)) implementiert werden?

Wir versuchen es mit einem signed Integer mit 16 Bit. Somit muss der Betrag des Stellgrößenwerts des P–Anteils kleiner gleich $2^{15} - 1$ sein. Das führt auf einen maximalen Betrag des Regelfehlers des Drehzahlreglers von

$$\frac{2^{15} - 1}{\lfloor k_{Rn} \cdot n_n \rfloor} = \frac{2^{15} - 1}{307} = 106.7 \; U/\min$$

Da wir dies als zu klein ansehen, versuchen wir es alternativ mit einem signed Integer mit 32 Bit. Jetzt muss der Betrag des Stellgrößenwerts des P–Anteils kleiner gleich $2^{31} - 1$ sein. Das führt auf einen maximalen Betrag des Regelfehlers von

$$\frac{2^{31} - 1}{\lfloor k_{Rn} \cdot n_n \rfloor} = \frac{2^{31} - 1}{307} = 6.99 \cdot 10^6 \; U/\min\,.$$

Dies ist sicherlich groß genug. Wenn wir den Betrag des Regelfehlers auf diesen Wert begrenzen, kann kein Überlauf im P–Anteil des Drehzahlreglers entstehen.

Laboraufgabe 7: Analyse des I–Anteils des Drehzahlreglers

Analysieren Sie die Implementierung des Drehzahlreglers (Datei *zPIawu_n_int.c*) und die Implementierung des Positionsreglers in der Datei *zP_pos_int.c*. Finden Sie die Werte der Skalierungsfaktoren n_I und n_p heraus und tragen Sie diese in das Blockschaltbild Teil c) der Abbildung 8 ein.

Laboraufgabe 8: Analyse des I–Anteils des Drehzahlreglers

Ermitteln Sie den absoluten und relativen Fehler des Parameters $k_I = k_{Rn} \dfrac{T}{T_{Nn}}$ durch die Integer–Realisierung. Ermitteln Sie dazu folgende Größen:

Hinweis: Bitte geben Sie alle Zahlenwerte mit 4 gültigen Stellen an. Evtl. müssen Sie bei den Rechnungen mit 10^3 skalieren, wenn die Zahlen auf Ihrem Taschenrechner mit zu wenigen gültigen Stellen dargestellt werden.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

Gleitkomma–Wert: $k_I = k_{Rn} \frac{T}{T_{Nn}} =$

Integer–Wert: $\hat{k}_I =$

Absoluter Fehler:

Relativer Fehler:

Wir haben beim P–Anteil des Drehzahlreglers entschieden, dass wir die internen Signale des Drehzahlreglers mit dem Datentyp int32 implementieren. Welchen maximalen Betrag darf der Regelfehler haben, damit es zu keinem Überlauf beim I–Anteil des Reglers kommt?

Maximaler Regelfehler für den I–Anteil:

Bei der Anlyse des P–Anteils des Drehzahlreglers wurde der maximale Regelfehler $6.99 \cdot 10^6$ $U/$ min ermittelt, damit kein Überlauf beim P–Anteil des Reglers entstehen kann. Welcher Wert ist damit jetzt final zu implementieren?

Ergebnis P- und I–Anteil zusammen:

In der Implementierung des Reglers wird der Regelfehler auf einen deutlich kleineren Wert begrenzt.

Laboraufgabe 9: Integer–Realisierung des Positionsreglers in Simulink

Implementieren Sie den Positionsregler mit den gegebenen Skalierungsfaktoren in der Datei *zP_pos_int.c*. Die Datei können Sie mit dem Matlab–Kommando `mex` kompilieren.

Kompilieren Sie natürlich auch die Datei *zPlawu_n_int.c* mit `mex`.

Laboraufgabe 10: Verifikation der Integer–Realisierungen der Regler in Simulink

Verwenden Sie das Simulink–Modell *Regelkreis_ZD_C_int.slx* und simulieren Sie zunächst die Sprungantwort des Regelkreises mit Drehzahlregelung und dann mit Positionsregelung und vergleichen Sie die Ergebnisse mit den Simulationsergebnissen mit dem reinen Simulinkmodell *Regelkreis_ZK.slx*. Falls größere Abweichungen auftreten, überprüfen Sie zunächst, ob Sie bei beiden Modellen dieselben Reglerparameter eingestellt haben und dann Ihre Skalierungsfaktoren.

Laboraufgabe 11: C–Programm in Integer–Arithmetik auf Mikrocontroller

Tragen Sie die Skalierungsfaktoren in den C–Code für den Mikrocontroller unter *Lab2_uC_integer/src/main.c* ein. Sie müssen in dieser Datei nur an den mit

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

ToDo markierten Stellen Änderungen durchführen.

Kompilieren Sie das Projekt und laden Sie es auf das Board. Sehen Sie sich das Regelverhalten für den Drehzahl- und den Positionsregler mit dem SoftScope an. Durch die Integer–Realisierung sollte das Regelverhalten praktisch unverändert bleiben.

Vervollständigen Sie die Tabelle 12 mit den Rechenzeiten der beiden Regler in Integer–Realisierung.

Zu dieser Aufgabe müssen Sie nichts abgeben.

Laboraufgabe 12: Laufzeiten der verschiedenen Mikrocontroller–Programme

Tragen Sie die gemessenen Werte der Laufzeiten der C–Programme für die beiden Betriebsarten der Regelung in die folgende Tabelle ein:

Reglerart ARM RP2040	C–Code mit float–Variablen	Autocode mit float–Variablen	C–Code mit Integer–Variablen
Drehzahlregelung			
Positionsregelung			

Hinweis: Die gemessenen Werte enthalten die Rechenzeit der Regler, der Stimuligenerierung für die sprungförmigen Sollwerte sowie die Datenübertragung des SoftScopes, die ebenfalls innerhalb der Reglerroutine implementiert wurden.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

5 Anhang: Analyse von Messdaten

5.1 Software-Oszilloskop

Interne Werte des Mikrocontrollerprogramms, z.B. die mit dem ADC gemessene Motordrehzahl, die Solldrehzahl oder die Istposition, können über die serielle Schnittstelle im Echtzeitbetrieb an den PC übertragen und in Matlab angezeigt werden. Dazu wird **im Mikrocontrollerprogramm** am Ende der Funktion `repeating_timer_callback()`, die im Abtastraster alle $T = 1000\mu sec$ ausgeführt wird, die Funktion

```
SoftScope((int16) groesse_1, (int16) groesse_2);
```

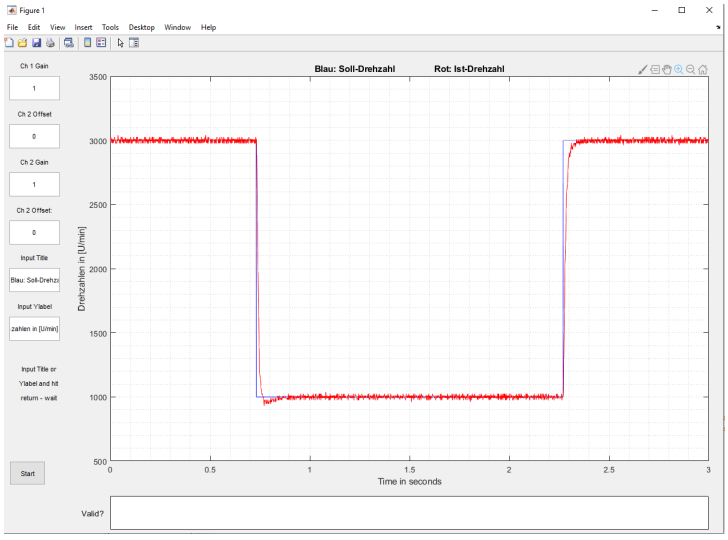
aufgerufen. Beachten Sie, dass die übertragenen Werte in 16bit Integerzahlen konvertiert werden. Für eine Übertragung von Gleitkommazahlen oder von mehr als zwei Werten innerhalb der Abtastzeit ist die serielle Schnittstelle (115kbit/sec) zu langsam.

Das Mikrocontrollerboard ist über eine USB–Schnittstelle mit dem PC verbunden werden. Über diese Schnittstelle wird das Board geflashed. Im Betrieb kann über diese Schnittstelle auch die Standardausgabe des Mikrocontrollers an den PC übertragen werden.

In der Matlab-Konsole muss die folgende Funktion für das Software-Oszilloskop ausgeführt werden:

```
SoftScope(0)     % listet vorhande Schnittstellen auf
SoftScope(25)    % erwartet Daten von serieller Schnittstelle 25
```

Die dargestellten Zeitsignale sind die vom Mikrocontroller-Programm gesendeten 16bit-Integerwerte. Das Bildschirmfenster zeigt einen Zeitbereich von einigen Sekunden an und wird laufend aufgefrischt. Um Details näher analysieren zu können, stoppen Sie das Oszilloskop über den Start/Stop-Button im Programmfenster und zoomen Sie im Bild mit den üblichen Matlab-Werkzeugen.

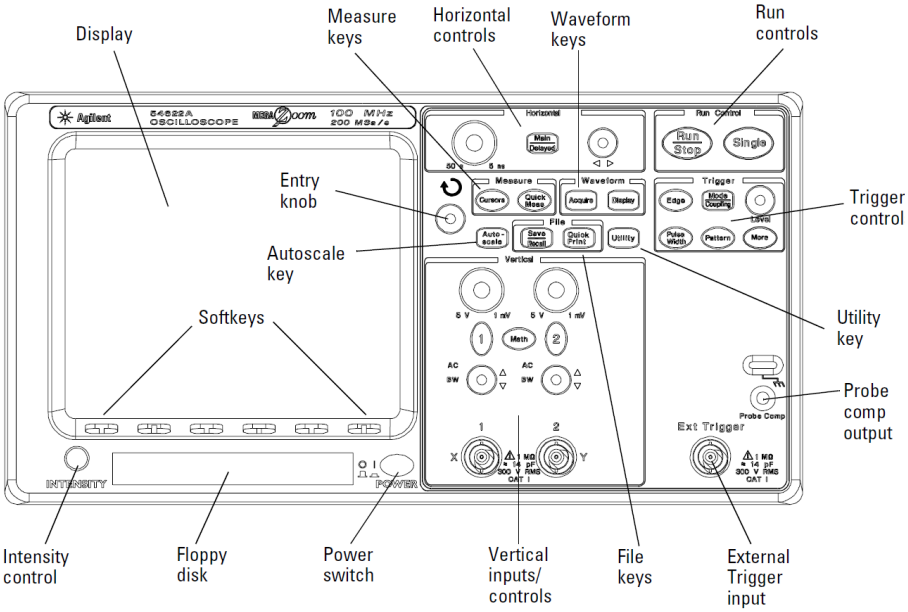


Achtung: Falls das Software-Oszilloskop hängen bleibt, schließen Sie das Fenster und starten Sie es neu. Eventuell müssen Sie anschließend `clear all` ausführen oder sogar Matlab neu starten.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

5.2 Bedienung des Digital–Speicher–Oszilloskops

Die Bedienung erfolgt über Tasten (**Hardkeys**) und Drehknöpfe (**Knobs**) auf der Vorderseite des Gerätes sowie Menütasten (**Softkeys**) unterhalb des Displays. Wenn eine Menütaste mehrere Optionen hat, wählt man diese über den Drehknopf **Entry knob**.



Grundeinstellungen:

- **Tastköpfe anschließen** und mit Messignalen verbinden. Zugehörige Kanäle über Hardkeys 1 und 2 einschalten.
- Hardkey **Autoscale** drücken: Stellt Kanalverstärkung und Trigger automatisch ein, funktioniert aber nur bei „schnellen“ Signalen, z.B. den TTL-Signalen des Positionsgebers. Für Messung von Sollwertsprüngen der Drehzahl mit dem Tachogebersignal nicht geeignet.
- Erneutes Drücken von Autoscale schaltet auf die vorige manuelle Einstellung zurück.

Kanalverstärkung und Zeitablenkung:

- Die **Verstärkung** kann für jeden Kanal an den großen Drehknöpfen im Bedienfeld Vertical eingestellt werden. Mit den darunterliegenden kleinen Drehknöpfen kann für jeden Kanal der **Offset**, d.h. die vertikale Lage, verstellt werden.
- Die **Zeitablenkung** wird mit dem großen Drehknopf im Bedienfeld Horizontal eingestellt.
- Die eingestellten Werte (V/Div bzw. s/Div) werden im Display in der oberen Zeile angezeigt.

Trigger:

- Hardkey **Edge** wählt man das Menü im Display aus. Mit Softkey **Rising ⇑ bzw. Falling ⇓** kann die steigende oder fallende Signalflanke zum Triggern ausgewählt werden. Mit den Softkeys 1 bzw. 2 wählt man den **Triggerkanal** aus. Der **Triggerpegel** kann über den Drehknopf Level im Bedienfeld Trigger eingestellt werden.
- Hardkey **Mode** wählt das Menü im Display aus. Über den Softkey Mode wählt man Auto oder Normal (für langsame Signale wie Sollwertsprünge besser geeignet). Bei **Coupling** sollte DC ausgewählt werden.

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann

6 Anhang: Bedienoberfläche des Mikrocontroller–Programms

Bedientasten:

- Taste SW07_1:
  Umschalten zwischen Drehzahlregelung (R), Positionsregelung (P) und Handbetrieb (M)
- Taste SW07_2:
  Umschalten zwischen automatischer Sollwerterzeugung (eingebauter Rechteckgenerator) oder Sollwertvorgabe über SW07_3/SW07_4.
- Taste SW07_3:
  z.B. Sollwert vergrößern
- Taste SW07_4:
  z.B. Sollwert verringern

Anzeige im LCD–Display obere Zeile:

- Im RPM–(Drehzahlregler)–Modus:
  R: w=Solldrehzahl e=Regelfehler (beide in U/min)
- Im Positionsregler–Modus:
  P: w=Sollposition e=Regelfehler (beide in Ticks)
- Im Manual–(Hand)–Modus:
  M: u=manuell vorgegebene Stellgröße nT: Tacho–Generator–Drehzahl (in U/min)

Anzeige im LCD–Display untere Zeile:

- Bedienhinweise zu den Tasten SW07_3, SW07_4 zur Sollwertvorgabe
- Nach *AutoStim* wird die Rechenzeit der Reglerroutine angezeigt
- Bedeutung der Drehzahldarstellungen
  - nT : Tachogenerator: Drehzahl vom analogen Messwert
  - nE : Encoder: Drehzahl vom Quadraturpositionsgeber

© Professoren W. Lindermeir, T. Rothermel, W. Zimmermann